

EXPLAIN ANALYZE 분석 참고 자료

본 문서는 Exqutor의 Adaptive Sampling 내부 동작을 EXPLAIN ANALYZE로 분석한 실제 출력을 정리한 것이다.

1. EXPLAIN ANALYZE 텍스트 출력 (BERN Baseline)

서버에서 실제 실행한 쿼리와 그 plan tree 출력이다.

쿼리: 1M subset 테이블에서 L2 거리 < 4.0인 행 수를 카운트

```
SET vector.sample_size = 385;

EXPLAIN ANALYZE
SELECT count(*)
FROM partsupp_deep_10_subset_1m
WHERE ps_embedding <-> '<query_vector> '::vector < 4.0;
```

출력:

```
Aggregate (cost=169274.09..169274.11 rows=1 width=8)
  (actual time=334.439..334.440 rows=1 loops=1)
  -> Sample Scan on partsupp_deep_10_subset_1m
      (cost=0.00..166671.77 rows=1040927 width=0)
      (actual time=0.289..334.382 rows=287 loops=1)
      Sampling: bernoulli ('0.028917'::real)
      Filter: ((ps_embedding <-> '<query_vector> '::vector) < '4'::double precision)
Planning Time: 0.740 ms
Execution Time: 699.023 ms
```

핵심 분석 포인트

항목	값	의미
Node Type	Sample Scan	Exqutor hook이 원래 Seq Scan을 Sample Scan으로 교체함
Sampling Method	bernoulli	BERNOULLI 샘플링 (블록 편향 없는 행 수준 균등 추출)
Sampling Rate	0.028917%	$\text{sample_size}(385) / \text{total_rows}(1M) \times 100$
Plan Rows	1,040,927	옵티마이저의 카디널리티 추정값 — hook이 반환한 값
Actual Rows	287	실제 실행 시 반환된 행 수
Planning vs Execution	0.74ms / 699ms	플래닝은 빠르지만 실행에서 샘플 스캔 비용 발생

핵심: Plan Rows (추정)와 Actual Rows (실제)의 괴리가 Q-error의 원천이다. 우리 연구는 이 괴리를 줄이기 위해 BERNOULLI를 Stratified Sampling(KM20)으로 교체한다.

2. EXPLAIN (FORMAT JSON) Plan Tree 구조

동일 쿼리의 JSON 포맷 출력으로, 실험 스크립트가 프로그램적으로 파싱하는 형태이다.

```
[
  {
    "Plan": {
      "Node Type": "Aggregate",
      "Strategy": "Plain",
      "Startup Cost": 169144.41,
      "Total Cost": 169144.42,
      "Plan Rows": 1,
      "Plan Width": 8,
      "Actual Startup Time": 322.384,
      "Actual Total Time": 322.384,
      "Actual Rows": 1,
      "Actual Loops": 1,
      "Plans": [
        {
          "Node Type": "Sample Scan",
          "Parent Relationship": "Outer",
          "Relation Name": "partsupp_deep_10_subset_1m",
          "Startup Cost": 0.00,
          "Total Cost": 166671.77,
          "Plan Rows": 989053,
          "Plan Width": 0,
          "Actual Startup Time": 0.204,
          "Actual Total Time": 322.327,
          "Actual Rows": 275,
          "Actual Loops": 1,
          "Sampling Method": "bernoulli",
          "Sampling Parameters": ["'0.028917'::real"],
          "Filter": "((ps_embedding <-> '... '::vector) < '4'::double precision)",
          "Rows Removed by Filter": 0
        }
      ]
    },
    "Planning Time": 0.671,
    "Execution Time": 686.071
  }
]
```

실험 스크립트의 파싱 로직

```
# EXPLAIN (ANALYZE, FORMAT JSON) 결과에서 핵심 지표 추출
plan = json_result[0]["Plan"]
scan_node = plan["Plans"][0] # Sample Scan 노드

hook_est = ... # 서버 로그(eLog)에서 추출 - authoritative metric
plan_rows = scan_node["Plan Rows"] # 989,053 (옵티마이저 추정)
actual_rows = scan_node["Actual Rows"] # 275 (실제)
sampling_method = scan_node["Sampling Method"] # "bernoulli"
sampling_rate = scan_node["Sampling Parameters"][0] # 0.028917%

# Q-error 계산
qerror = max(hook_est / actual_count, actual_count / hook_est)
```

주의: `Plan Rows` 는 Stratified 쿼리에서 stratum LIMIT 상수(20)를 반환하는 구조적 문제가 있어, `hook_est` (서버 로그 기록값)를 authoritative metric으로 사용한다. 이 측정 원칙은 Phase 6 Step 4에서 확립되었다.

3. vector.c 소스코드 — Hook Trigger + Sampling 교체 지점

3-1. Hook Trigger (vector.c L287)

Exquotor의 `planner_hook` 이 쿼리를 가로채는 진입점이다.

```
// vector.c L284~300 (현재 서버 - 수정 적용됨)
table_count = 0;
count_total_tables(parse, &table_count);

if (table_count >= 1) // ← 원본: table_count > 2 (JOIN 2테이블 이상만)
{
    // 수정: >= 1 (단일 테이블도 hook 진입)
    MemoryContext oldCtx = MemoryContextSwitchTo(TopMemoryContext);
    ordering_needed = true;
    original_query = (Query *)copyObject(parse);
    original_query_string = (char *)query_string;
    original_cursorOptions = cursorOptions;
    original_boundParams = boundParams;
    MemoryContextSwitchTo(oldCtx);
}
```

항목	원본	수정
조건	<code>table_count > 2</code>	<code>table_count >= 1</code>
의미	JOIN이 있는 multi-table 쿼리만 hook	단일 테이블 벡터 쿼리도 hook 진 입
발견	EXPLAIN ANALYZE에서 단일 테이블 쿼리가 Seq Scan으로 실행됨을 확인	
근거	원논문 시나리오는 TPC-H JOIN 쿼리 전제 → 단일 테이블은 명시되지 않은 사 각지대	

3-2. Sampling 방식 (vector.c L940)

Hook이 카디널리티를 추정할 때 사용하는 내부 샘플링 쿼리이다.

```
// vector.c L938~944
appendStringInfo(&query,
    "SELECT COUNT(*)::float FROM "
    "(SELECT %s FROM %s TABLESAMPLE BERNOULLI(%f)) p "
    "WHERE p.%s %s '%s' < %f",
    vector_column_name,      // ps_embedding
    vector_table_name,       // partsupp_deep_10_subset_1m
    sample_ratio,            // 0.028917 (= 385/1M × 100)
    vector_column_name,      // ps_embedding
    distance_function,       // <->
    vector_str,              // query vector
    range_distance_value);   // 4.0 (D_target)
```

항목	설명
BERNOULLI(0.029%)	전체 테이블에서 행 수준 균등 추출 (385행 기대)
문제점	데이터가 공간적으로 쏠려 있을 때, 균등 추출은 밀집 영역을 과소/과대 대표
우리의 개선	BERNOULLI → KM20 Stratified Sampling (k-means 20 파티션 기반 층화 추출)

관련 실험 결과 요약

비교	selectivity	diff%	p-value	의미
KM20 vs BERN	s=0.500	+1.64%	p<0.004	5-seed 평균, CI [1.25, 2.02]
KM20 vs BERN	s=0.010	+8.93%	p<0.001	좁은 범위에서 효과 극대화
RANDOM20 vs BERN	s=0.010	-10.67%	—	무작위 파티션은 오히려 악화

결론: 공간 인식(KM20) 파티션이 핵심이며, 이는 EXPLAIN ANALYZE의 Plan Rows 추정 정확도를 selectivity-dependent하게 개선한다.